

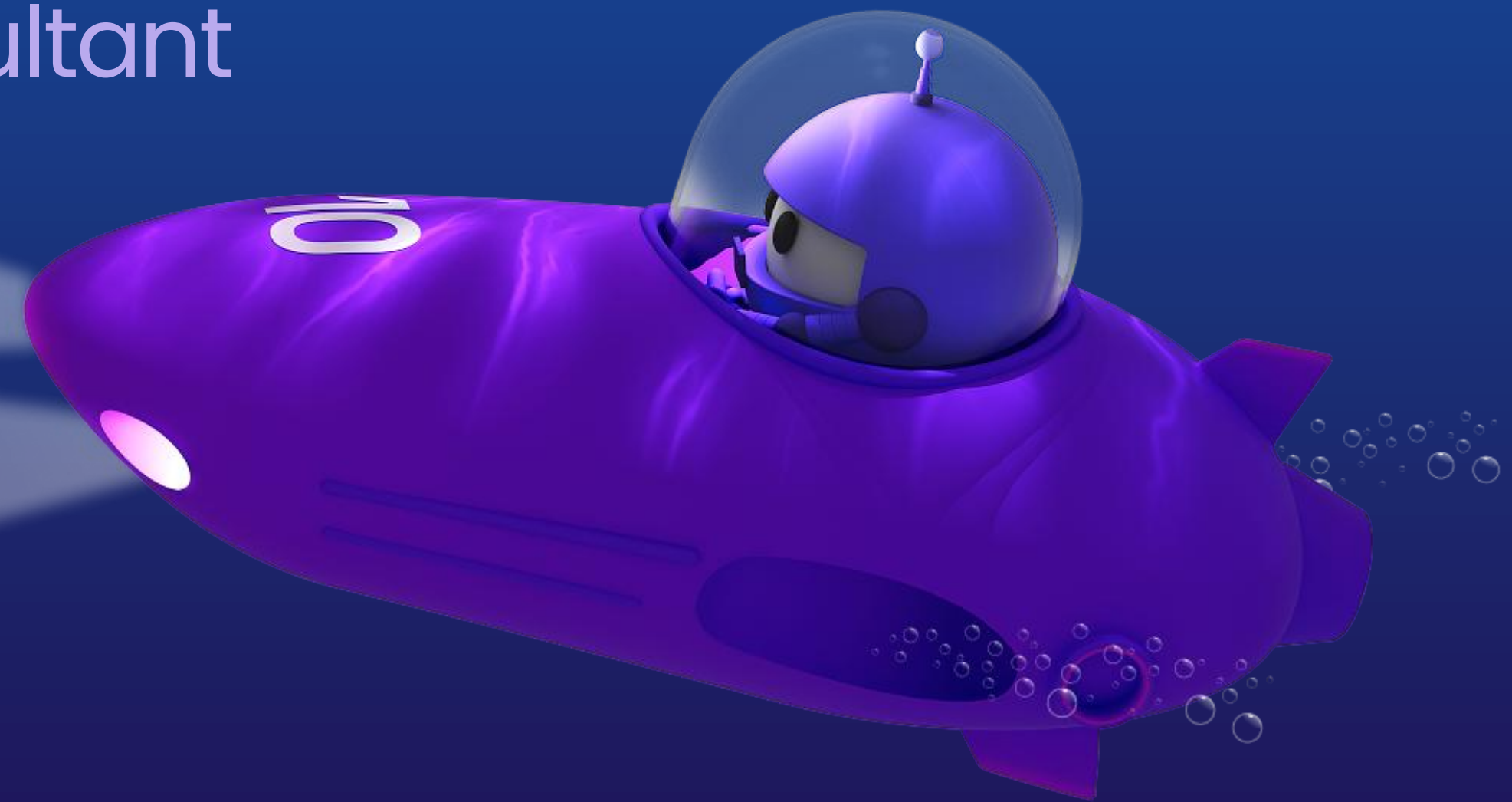
ASP.NET Core e .NET 10:

.NET

le novità che potenziano lo sviluppo web

Andrea Dottor

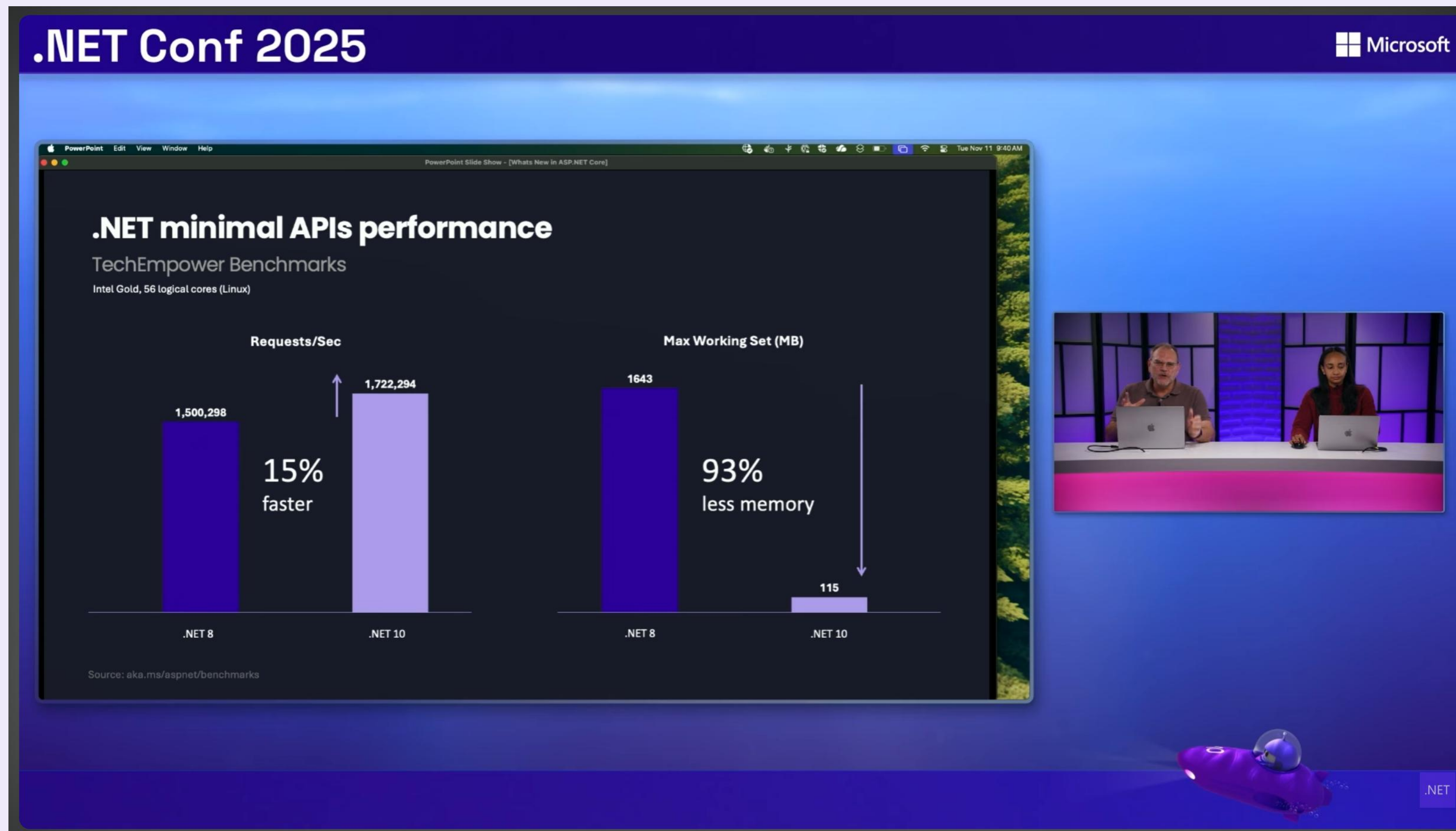
andrea@dottor.net – Senior Developer & Consultant
Microsoft MVP Developer Technologies



Sponsor



.NET minimal APIs performance



.NET 9 performance (da .NET Conf 2024)

The fastest ASP.NET Core yet!

↑ **15%** throughput*

↓ **93%** memory usage on high core-count machines*

↓ **25%** startup time for Blazor WebAssembly**

WebSocket message compression for Blazor Server

Precompression and improved caching for static web assets

* TechEmpower JSON Min APIs, Intel Gold 56 cores (logical) Linux

** Lighthouse

Blazor performance

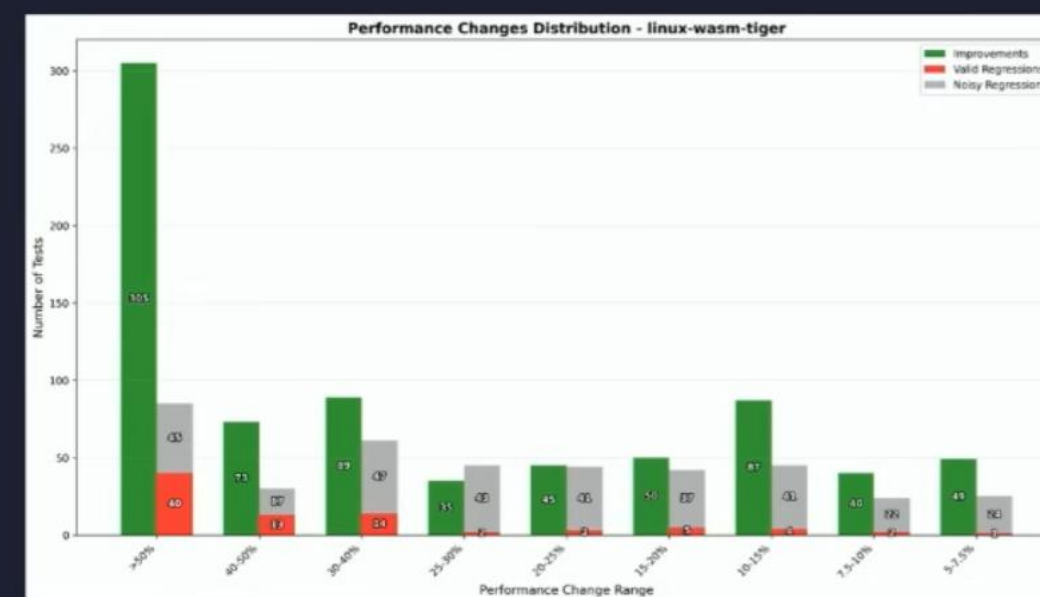
.NET Conf 2025



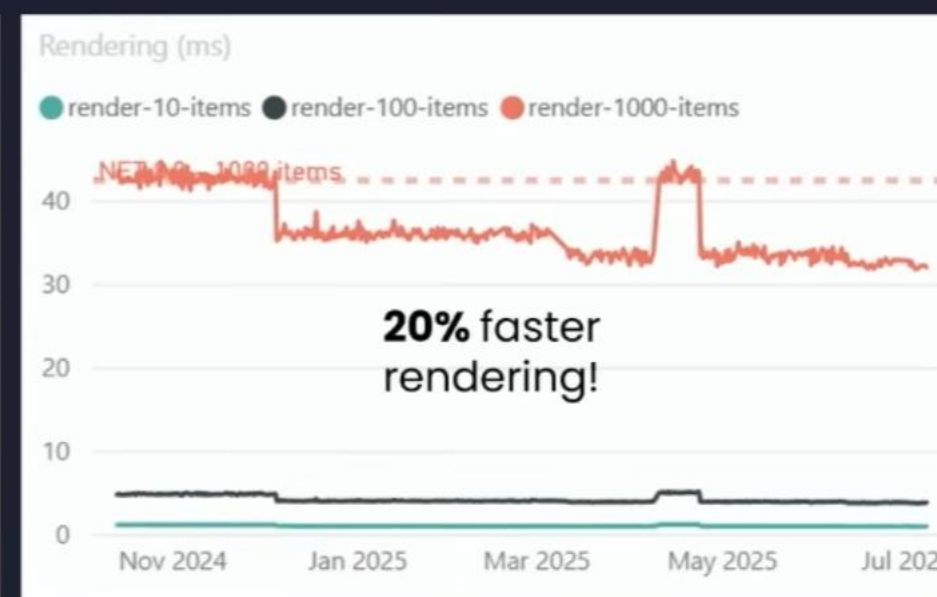
Improved performance

Startup & runtime

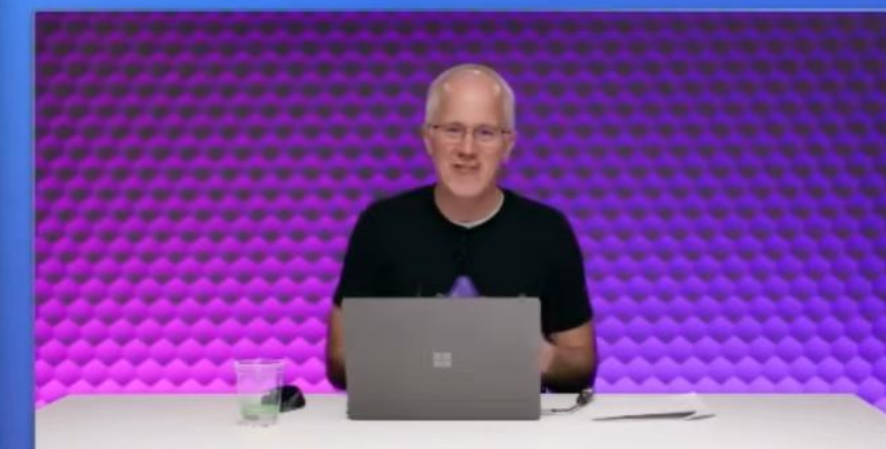
- Blazor framework scripts fingerprinted, compressed, and cached
- Preload Blazor WebAssembly resources
- Faster runtime execution & rendering



<https://github.com/dotnet/performance/blob/main/reports/net9to10/README.md>



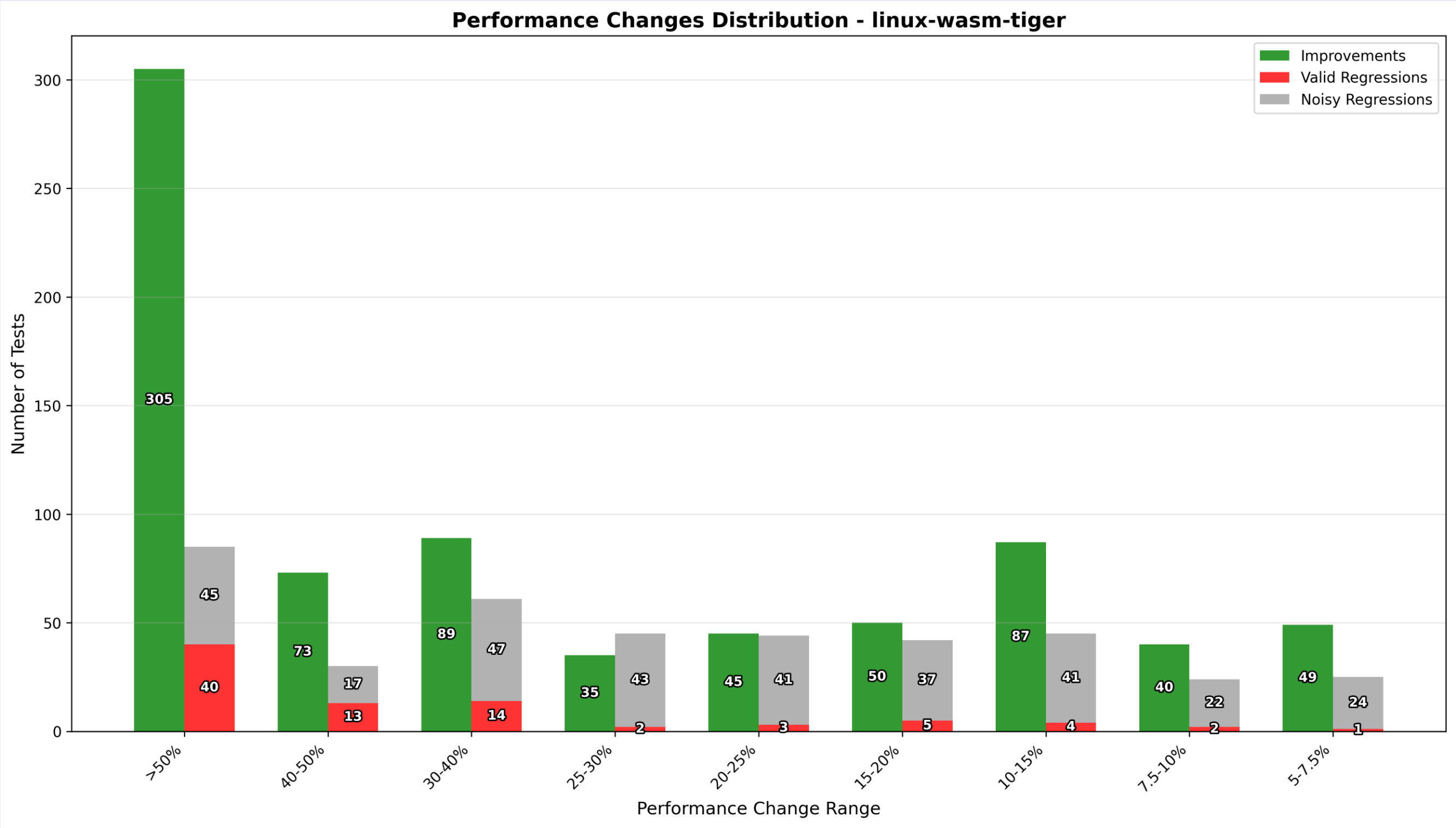
<https://aka.ms/aspnet/benchmarks>



aka.ms/dotnet/maui/2025

.NET

Blazor performance



Supporto al Top-Level Domain .localhost

Il dominio di primo livello (TLD) .localhost è definito in RFC2606 e RFC6761 come riservato a scopo di test e disponibile per gli utenti da utilizzare localmente come farebbero con qualsiasi altro nome di dominio.

- I moderni browser risolvono già automaticamente qualsiasi nome *.localhost nell'indirizzo di loopback IP (127.0.0.1/::1)
- `https://localhost:6789` == `https://miaapp.dev.localhost:6789`

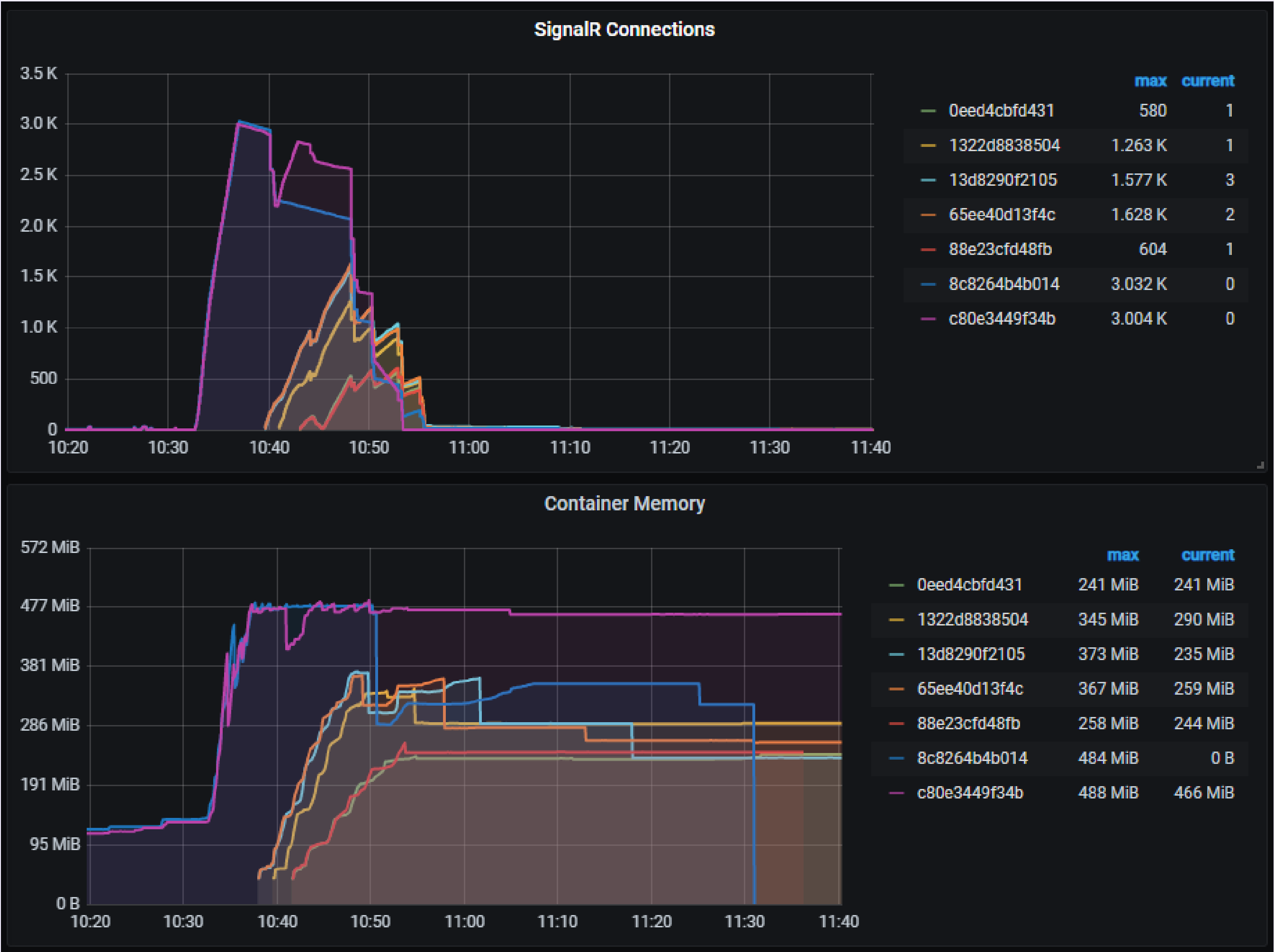
Miglioramento gestione memory pool

I pool di memoria usati da Kestrel, IIS e HTTP.sys ora sono in grado di ridurre la memoria occupata quando l'applicazione è inattiva o ha meno carico.

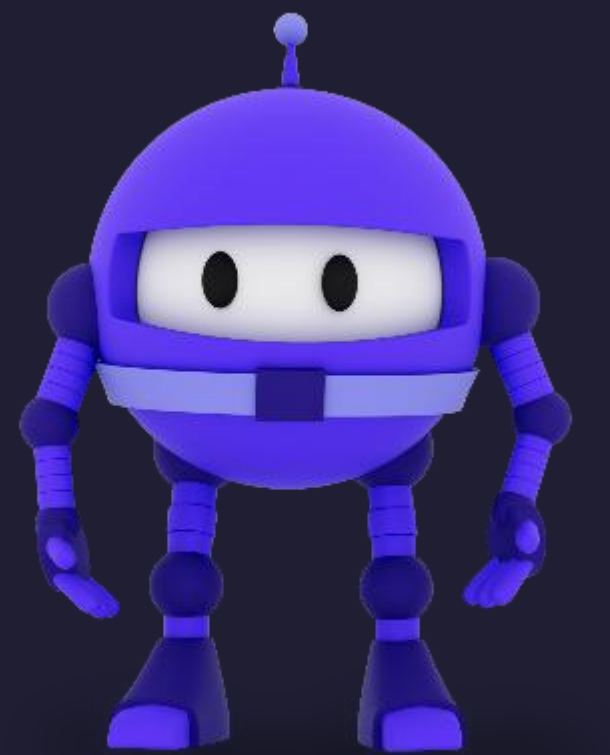
La funzione viene eseguita automaticamente e non deve essere abilitata o configurata manualmente.

- How Microsoft Developers Use AI in Real-World Coding | BRK103
David Fowler & Stephen Toub
<https://youtu.be/gieL0bxyTUU?si=21Xp7xNME6CGFuQu>

Miglioramento gestione *memory pool*



Microsoft.Extensions.Validation



Microsoft.Extensions.Validation

Validazione tramite DataAnnotation migliorata, che include ora il supporto agli oggetti annidati e alle collezioni

- Per attivare la nuova funzionalità:
 - Richiamare l'extension method **AddValidation** nel file Program.cs.
 - Annota il model principale con l'attributo **[ValidatableType]**
- Possibilità di validazioni complesse implementando **IValidatableObject**

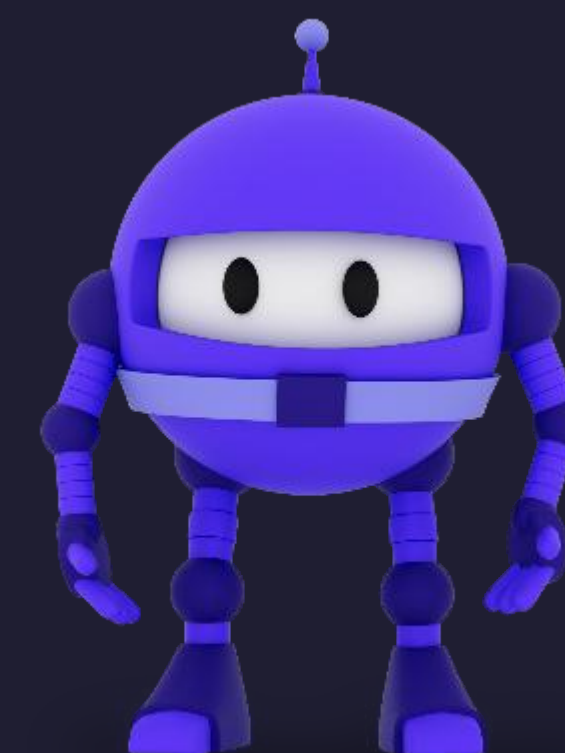
Minimal APIs



Minimal APIs novità

- Il supporto per la validazione nelle Minimal APIs è ora disponibile, e supporta
 - Query
 - Header
 - Request body
- Validazione anche per i Records type
- Supporto a Server-Sent Events (SSE)
- OpenAPI 3.1
- Supporto ai commenti in XML per OpenAPI

DEMO



Blazor



NavigationException in SSR

- La chiamata a `NavigationManager.NavigateTo` durante il rendering statico lato server (SSR statico) genera un'eccezione `NavigationException`, interrompendo l'esecuzione prima di essere convertita in un redirect. Ciò può causare confusione durante il debug ed è incoerente con il comportamento di rendering interattivo, in cui il codice dopo `NavigateTo` continua a essere eseguito normalmente.
- In .NET 10 è possibile impostare la proprietà `<BlazorDisableThrowNavigationException>` di MSBuild a `true` nel file di progetto dell'app per evitare di generare l'eccezione durante il SSR.
 - I progetti .NET 10 Blazor Web App impostano per default la proprietà a `true`.

Observability & diagnostics

- Nuove metriche specifiche per Blazor
- Miglioramento del tracing per Blazor Server
- Diagnostic tools per Blazor WebAssembly

Router: parametro NotFoundPage

Blazor ora offre un modo migliorato per visualizzare una pagina “NotFound” quando si tenta di accedere a una pagina inesistente.

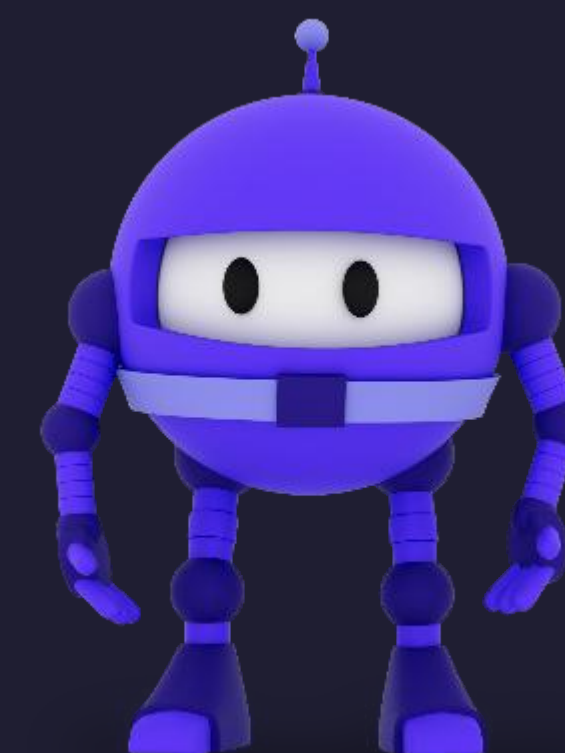
È possibile specificare quale pagina mostrare quando viene invocato `NavigationManager.NotFound` passando il tipo di pagina al componente Router tramite il parametro `NotFoundPage`.

Componente ReconnectModal.razor

E' presente ora un componente ReconnectModal.razor che permette di personalizzare il popup di riconnessione nelle applicazioni con interattività server.

- Nuove funzionalità dell'interfaccia di riconnessione:
 - Oltre a indicare lo stato di riconnessione impostando una specifica classe CSS sull'elemento dell'interfaccia, il nuovo evento components-reconnect-state-changed viene emesso quando lo stato di riconnessione cambia.
 - Il codice può distinguere meglio le diverse fasi del processo di riconnessione grazie al nuovo stato di riconnessione "retrying", segnalato sia tramite la classe CSS che tramite il nuovo evento.

DEMO



PersistentState

Con .NET 10 puoi dichiarare facilmente lo stato da persistere in componenti e servizi tramite l'attributo **[PersistentState]**. Lo stato viene gestito in automatico dal servizio **PersistentComponentState** durante il prerendering e ripristinato al primo rendering interattivo.

Persistenza Circuit state

Durante il rendering lato server, Blazor Web Apps può ora rendere persistente lo stato della sessione (circuito) di un utente quando la connessione al server viene persa per un periodo di tempo prolungato o sospesa in modo proattivo, purché non venga attivato un aggiornamento dell'intera pagina.

Ciò consente agli utenti di riprendere la sessione senza perdere il lavoro non salvato nei seguenti scenari:

- Limitazione delle schede del browser
- Utenti di dispositivi mobili che cambiano app
- Interruzioni di rete
- Gestione proattiva delle risorse (sospensione dei circuiti inattivi)
- Navigazione migliorata

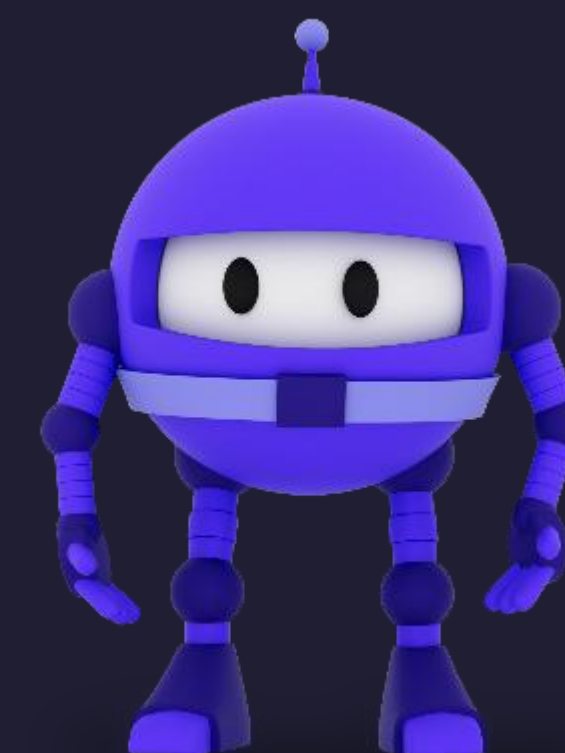
Pausa e ripristino delle sessioni (circuiti)

- Puoi ora sospendere e riprendere i circuiti per andare a migliorare la scalabilità di un'app.
 - La sospensione di un circuito memorizza i dettagli sul circuito nella memoria del browser lato client e rimuove il circuito, liberando le risorse del server.
 - La ripresa del circuito stabilisce un nuovo circuito e lo inizializza utilizzando il PersistentState.
- Da JavaScript:
 - Chiama *Blazor.pauseCircuit()* per mettere in pausa un circuito.
 - Chiama *Blazor.resumeCircuit()* per riprendere un circuito.

Trigger state persistence from the server #62327

- <https://github.com/dotnet/aspnetcore/issues/62327>

DEMO



Autenticazione e autorizzazione



Nuove metriche per Autenticazione e Autorizzazione

Aggiunta di nuove metriche per gli eventi di **Authentication** e **Authorization**.

- Authentication:
 - Authenticated request duration
 - Challenge count
 - Forbid count
 - Sign in count
 - Sign out count
- Authorization:
 - Count of requests requiring authorization

Nuove metriche per ASP.NET Core Identity

Metriche in Microsoft.AspNetCore.Identity:

- aspnetcore.identity.user.create.duration
- aspnetcore.identity.user.update.duration
- aspnetcore.identity.user.delete.duration
- aspnetcore.identity.user.check_password_attempts
- aspnetcore.identity.user.generated_tokens
- aspnetcore.identity.user.verify_token_attempts
- aspnetcore.identity.sign_in.authenticate.duration
- aspnetcore.identity.sign_in.check_password_attempts
- aspnetcore.identity.sign_in.sign_ins
- aspnetcore.identity.sign_in.sign_outs
- aspnetcore.identity.sign_in.two_factor_clients_remembered
- aspnetcore.identity.sign_in.two_factor_clients_forgotten

WebAuthN & Passkeys

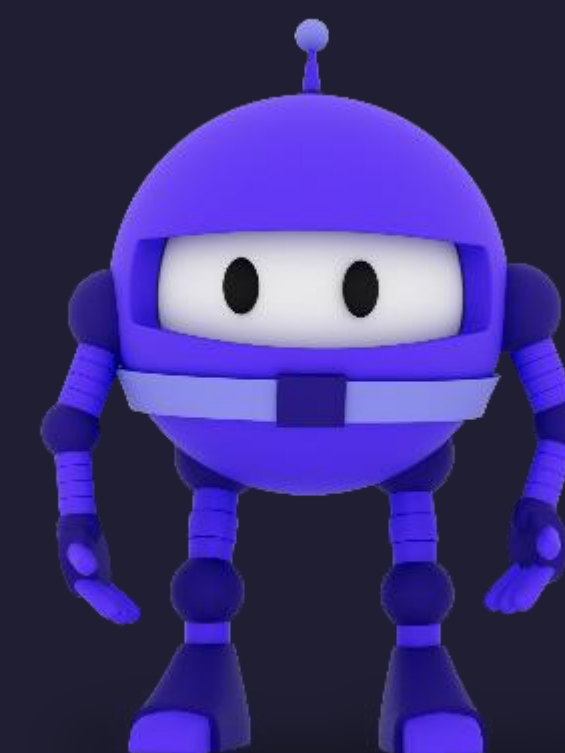
Il template di progetto sono stati aggiornati aggiungendo il supporto alle Passkey

- Funzionalità integrata in ASP.NET Core Identity

I Passkey sono credenziali crittografiche che sostituiscono le password tradizionali.

- Sono resistenti al phishing, facili da usare e sicuri per progettazione.
- Un passkey è composto da una coppia di chiavi pubblico-privata, associata a un account e a un'origine specifici.
 - La chiave privata è conservata in un Authenticator.
 - L' Authenticator richiede la verifica dell'utente prima di consentire l'autenticazione.

DEMO



Migrazione a .NET 10



Migrazione ASP.NET Core da .NET 9 a .NET 10

- Aggiornamento della versione del .NET SDK in global.json
- Aggiornamento del target framework
- Aggiornamento dei packages utilizzati
- Implementazione passkeys in ASP.NET Core Blazor Web Apps*
 - Aggiornamneto dello schema utilizzato per Identity: nel Program.cs, specificare di utilizzare la versione 3, che include il supporto alle passkey
 - Creare ed eseguire la migrazione
 - Importare/creare componenti e classi necessari al passkey
 - <https://learn.microsoft.com/en-us/aspnet/core/security/authentication/passkeys/blazor>

*Nel file di progetto delle applicazioni Blazor, settare *BlazorDisableThrowNavigationException* a true

Breaking changes in .NET 10

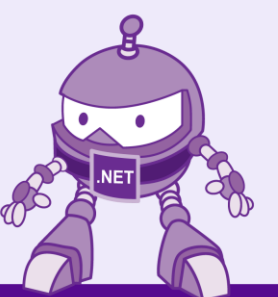
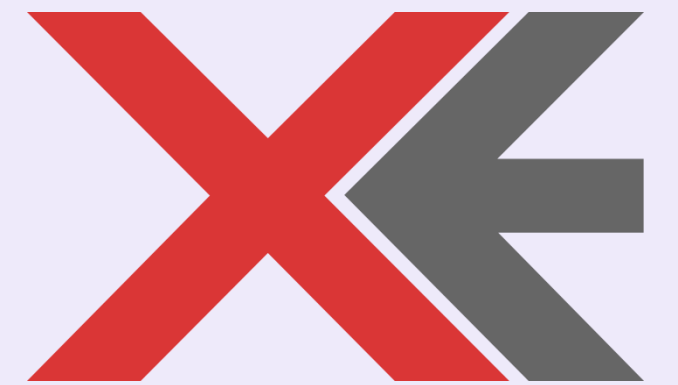
- Cookie login redirects disabled for known API endpoints
- Deprecation of WithOpenApi extension method
- Exception diagnostics suppressed when TryHandleAsync returns true
- `IActionContextAccessor` and `ActionContextAccessor` are obsolete
- `IncludeOpenAPIAnalyzers` property and MVC API analyzers are deprecated
- `IPNetwork` and `ForwardedHeadersOptions.KnownNetworks` are obsolete
- `Microsoft.Extensions.ApiDescription.Client` package deprecated
- Razor run-time compilation is obsolete
- `WebHostBuilder`, `IWebHost`, and `WebHost` are obsolete
- ...

Thank you



Contatti

```
{  
  name:      Andrea Dottor  
  email:     andrea@dottor.net  
  bsky:      @andrea.dottor.net  
  linkedin:  @andreadottor  
}
```



il podcast